PIXEL TRADING FIRM

FRONTEND IMPLEMENTATION PLAN

World Space: 2520 × 1080 | Background: office_rpg_map_extended.png

Canvas Engine + React Overlay + WebSocket + Coordinate Map

1. SYSTEM ARCHITECTURE

Three layers stacked on top of each other. Each layer has a single responsibility and never crosses into another layer's domain.

```
┌──────────────────────────────────────────────────────────┐
│  LAYER 3 — HTML OVERLAY  (React + Tailwind)              │
│  Taskbar, data panels, kill switch, inbox, DEV MENU      │
│  Position calculated from world coords every RAF tick    │
├──────────────────────────────────────────────────────────┤
│  LAYER 2 — DYNAMIC CANVAS  (HTML5 Canvas + RAF loop)     │
│  Agent sprites, animations, effects, overlays, cards     │
│  All effects reference coordinate map constants          │
├──────────────────────────────────────────────────────────┤
│  LAYER 1 — STATIC BACKGROUND  (PNG image)                │
│  office_rpg_map_extended.png  2520 × 1080                │
│  Drawn once via ctx.drawImage(), never touched again     │
└──────────────────────────────────────────────────────────┘
```

1.1 Coordinate Constants Module

Every pixel coordinate from the coordinate map lives in a single constants file. No magic numbers anywhere else in the codebase. All animations and effects import from here.

```
// coords.ts — master coordinate registry
export const WORLD = { W: 2520, H: 1080 }

export const AGENT_POSITIONS = {
  1:  { x: 1590, y: 210,  name: 'Researcher'         },
  2:  { x: 150,  y: 690,  name: 'OnChainAnalyst'     },
  3:  { x: 390,  y: 690,  name: 'WalletTracker'      },
```

```
  4:  { x: 990,  y: 690,  name: 'IntelAgent'         },
  5:  { x: 1170, y: 450,  name: 'SignalAggregator'   },
  6:  { x: 270,  y: 210,  name: 'MacroSentinel'      },
  7:  { x: 1350, y: 690,  name: 'RiskManager'        },
  8:  { x: 270,  y: 510,  name: 'TradingBot'         },
  9:  { x: 2220, y: 450,  name: 'PerformanceAnalyst' },
}

export const LOCATIONS = {
  AGGREGATOR_TABLE:  { x: 1158, y: 339 },
  SENTINEL_WALL:     { x: 373,  y: 62  },
  MAIN_DOORWAY:      { x: 653,  y: 367 },
  ARCHIVE_BIN:       { x: 875,  y: 352 },
  SERVER_RACK_1:     { x: 820,  y: 67  },
  SERVER_RACK_2:     { x: 719,  y: 92  },
}

// Grid system: X = gx * 60 + 30, Y = gy * 60 + 30
export const GRID_SIZE = 60
```

2. GAME LOOP & CANVAS SETUP

2.1 Canvas Initialization

The canvas is sized to the container in logical pixels, DPR-corrected for Retina displays. The background image is drawn once per resize and cached — not redrawn every frame.

```
// OfficeCanvas.tsx
const bgImageRef = useRef<HTMLImageElement | null>(null)
const bgCacheRef = useRef<HTMLCanvasElement | null>(null)  // pre-rasterized BG

function prerenderBackground(img: HTMLImageElement, w: number, h: number) {
  const off = document.createElement('canvas')
  off.width = w; off.height = h
  const ctx = off.getContext('2d')!
  ctx.imageSmoothingEnabled = false
  ctx.drawImage(img, 0, 0, w, h)  // scale to canvas device pixels
  bgCacheRef.current = off
}

// Render loop — background drawn from cache, not from original image:
function render(ctx: CanvasRenderingContext2D) {
  ctx.clearRect(0, 0, canvas.width, canvas.height)
  if (bgCacheRef.current) {
    ctx.drawImage(bgCacheRef.current, offsetX, offsetY, mapW, mapH)
  }
  // ... draw dynamic elements on top
}
```

2.2 World-to-Screen Transform

All world coordinates from the coordinate map are converted to screen coordinates each frame using the current zoom and pan. This single function is used by every animation and effect renderer.

```
// Transform world coords (2520×1080 space) to canvas device pixels
function worldToScreen(wx: number, wy: number): { sx: number; sy: number } {
  return {
    sx: Math.round(offsetX + wx * zoom),
    sy: Math.round(offsetY + wy * zoom),
  }
}

// Example: signal card flying from Agent 1 desk to Aggregator table:
const from = worldToScreen(AGENT_POSITIONS[1].x, AGENT_POSITIONS[1].y)
const to   = worldToScreen(LOCATIONS.AGGREGATOR_TABLE.x,
LOCATIONS.AGGREGATOR_TABLE.y)
// Tween between from and to using lerp over animation duration
```

3. AGENT SPRITE SYSTEM

3.1 Character FSM — States

State	Visual	Trigger
IDLE	Static seated pose, monitor dim	No active signal or task
ACTIVE	Typing animation (2-frame), monitor glow ON	Agent processing — any tool running
SIGNAL_FOUND	Stand up frame, memo sprite spawns	candidate_detected / narrative_detected
STAMPING_GREEN	Green stamp animation over document	token_cleared
STAMPING_RED	Red stamp, document slides to archive	signal_killed
URGENT	Jump frame, URGENT envelope spawns	wallet_buy_detected (Agent 3 only)
HOLD_ACTIVE	Standing, arms raised frame	tier4_triggered (Agent 6 only)
EXECUTING	Terminal flash, all screens green	trade_executed (Agent 8 only)
TP_HIT	Celebrate frame, confetti particles	take_profit_hit (Agent 8 only)

STOPPED_OUT	Slump frame, red screen flash	stop_loss_hit (Agent 8 only)
REPORTING	Walk to printer frame, report sprite	weekly_report_due (Agent 9 only)

3.2 Monitor Glow Effect

Each agent desk has a monitor glow rendered as a radial gradient on the canvas, positioned relative to the agent's world coordinate. Color changes based on agent state.

```
function renderMonitorGlow(ctx, agentId, state, zoom, offsetX, offsetY) {
  const pos = AGENT_POSITIONS[agentId]
  const { sx, sy } = worldToScreen(pos.x, pos.y - 20)  // above agent = monitor
  const radius = 35 * zoom

  const color = state === 'HOLD_ACTIVE'  ? 'rgba(255, 60,  60,  0.45)'
              : state === 'URGENT'        ? 'rgba(255, 180, 0,   0.45)'
              : state === 'ACTIVE'        ? 'rgba(0,   255, 136, 0.30)'
              : state === 'EXECUTING'     ? 'rgba(0,   200, 255, 0.45)'
              :                             'rgba(0,   255, 136, 0.08)'  // idle
dim

  const grd = ctx.createRadialGradient(sx, sy, 0, sx, sy, radius)
  grd.addColorStop(0, color)
  grd.addColorStop(1, 'rgba(0,0,0,0)')
  ctx.fillStyle = grd
  ctx.fillRect(sx - radius, sy - radius, radius * 2, radius * 2)
}
```

4. SIGNAL AGGREGATOR TABLE

The most visually complex element. All card effects are canvas sprites positioned relative to LOCATIONS.AGGREGATOR_TABLE {1158, 339}. The static table in the background image is purely decorative — all interactive elements float on top.

4.1 Signal Card Data Structure

```
interface SignalCard {
  signal_id:    string
  token_name:   string
  sources:      number[]      // which agent IDs have contributed
  score:        number        // current composite score
  age_ms:       number        // ms since first signal arrived
```

```
  state:        'flying' | 'on_table' | 'stacking' | 'combining'
               | 'outbound' | 'expiring' | 'killed'
  x:            number        // current world X (animated)
  y:            number        // current world Y (animated)
  targetX:      number        // destination world X
  targetY:      number        // destination world Y
  progress:     number        // 0→1 tween progress
  stackIndex:   number        // visual stack depth
}
```

4.2 Card Lifecycle — Full Animation Sequence

```
// 1. ARRIVAL — card spawns at sending agent's desk, flies to table
//    Origin:      AGENT_POSITIONS[agentId]
//    Destination: AGGREGATOR_TABLE + small random offset for stacking
//    Duration:    800ms, ease-out curve
//    Visual:      small colored rectangle (12×8px world space)
//                 color matches sending agent's status color

// 2. ON TABLE — card rests, score badge floats above
//    Score badge: white text on dark pill, updates live as more sources confirm
//    Decay visual: card opacity reduces 15% every 15 minutes

// 3. STACKING — second confirmation arrives for same token
//    New card flies in, lands slightly offset from existing card
//    Stack grows visually thicker with each confirmation
//    Confidence score badge brightens and increases

// 4. CONFLUENCE REACHED — score >= 8.0
//    All cards for token pulse white simultaneously (120ms)
//    Combine animation: cards slide together to single point (300ms)
//    Thick combined file sprite appears
//    STAMP animation plays (green stamp descends, bounces)
//    File slides to AGENT_POSITIONS[6] (MacroSentinel) — 600ms

// 5. EXPIRY — 45 min timer expires without confluence
//    Cards fade to 20% opacity over 500ms
//    Slide toward ARCHIVE_BIN {875, 352} — 700ms ease-in
//    Small dust particle burst on arrival at bin

// 6. KILLED — Agent 2 rejected the token
//    Red X sprite overlays the card immediately
//    Card shakes (3px left-right, 4 cycles, 200ms)
//    Slides to ARCHIVE_BIN — 400ms fast
```

4.3 Table State Renderer

```
function renderAggregatorTable(ctx, cards, zoom, offsetX, offsetY) {
  for (const card of cards) {
    const { sx, sy } = worldToScreen(card.x, card.y)
    const w = 12 * zoom
    const h = 8  * zoom

    // Card body
    ctx.fillStyle = card.state === 'killed'   ? '#FF4444'
                  : card.state === 'outbound'  ? '#00FF88'
                  : AGENT_COLORS[card.sources[0]] ?? '#AAAAAA'
    ctx.globalAlpha = card.state === 'expiring' ? 0.2 : 1.0
    ctx.fillRect(sx - w/2, sy - h/2, w, h)

    // Stack depth indicator (each layer offset by 1px)
    for (let i = 1; i < card.sources.length; i++) {
      ctx.fillStyle = AGENT_COLORS[card.sources[i]] ?? '#888888'
      ctx.fillRect(sx - w/2 + i*zoom, sy - h/2 - i*zoom, w, h)
    }

    // Score badge
    if (card.state === 'on_table' || card.state === 'stacking') {
      renderScoreBadge(ctx, card.score, sx, sy - h, zoom)
    }

    ctx.globalAlpha = 1.0
  }
}
```

5. MACRO SENTINEL VISUAL SYSTEM

Agent 6 sits at {270, 210}. The Sentinel Wall display is at {373, 62}. All effects are canvas overlays on top of the static background — the background image never changes.

5.1 Sentinel Wall — Live BTC/SOL Display

Two small chart sprites rendered at SENTINEL_WALL {373, 62}. These are mini line charts drawn directly on canvas, updated every 60 seconds from the WebSocket feed. Threshold lines are drawn as colored horizontal lines overlaid on the chart area.

```
function renderSentinelWall(ctx, btcData, solData, macroState, zoom, offsetX,
offsetY) {
  const { sx, sy } = worldToScreen(LOCATIONS.SENTINEL_WALL.x,
LOCATIONS.SENTINEL_WALL.y)
  const chartW = 80 * zoom
  const chartH = 40 * zoom
```

```
  // BTC chart (left panel)
  renderMiniChart(ctx, btcData, sx - chartW - 5*zoom, sy, chartW, chartH,
'#F7931A')

  // SOL chart (right panel)
  renderMiniChart(ctx, solData, sx + 5*zoom, sy, chartW, chartH, '#9945FF')

  // Threshold lines
  const warn  = getThresholdY(chartH, -0.03)  // -3% warning
  const hold  = getThresholdY(chartH, -0.05)  // -5% hold

  // Warning line — yellow, dashed
  ctx.setLineDash([3*zoom, 2*zoom])
  ctx.strokeStyle = macroState === 'WARNING' ? '#FFCC44' : 'rgba(255,204,68,0.4)'
  ctx.beginPath()
  ctx.moveTo(sx - chartW - 5*zoom, sy + warn)
  ctx.lineTo(sx + chartW + 5*zoom, sy + warn)
  ctx.stroke()

  // Hold line — red, solid
  ctx.setLineDash([])
  ctx.strokeStyle = macroState === 'HOLD' ? '#FF4444' : 'rgba(255,68,68,0.4)'
  // ... same pattern for hold line
  ctx.setLineDash([])
}
```

5.2 Tier 4 Hold — Full Visual Sequence

```
// TRIGGER: tier4_triggered WebSocket event received

// STEP 1 — Red room tint (immediate, 200ms fade-in)
//   ctx.fillStyle = 'rgba(255, 0, 0, 0.15)'
//   ctx.fillRect(0, 0, canvas.width, canvas.height)
//   Stored as holdOverlayAlpha, animated 0 → 0.15 over 200ms

// STEP 2 — HOLD sign drops (300ms after trigger)
//   Sprite: hold_sign.png (pre-loaded)
//   Start position: worldToScreen(1260, -80)  — above visible area
//   End position:   worldToScreen(1260, 280)  — center of office
//   Animation: ease-out bounce, 500ms
//   Chains visible: two short vertical lines above sign sprite

// STEP 3 — MacroSentinel character state → HOLD_ACTIVE
//   Character switches to standing/arms-raised sprite frame
//   Monitor glow → red rgba(255,60,60,0.45)

// STEP 4 — Signal cards on Aggregator table freeze
//   All card.state = 'frozen' — tween progress pauses
//   Cards get a blue-grey tint overlay
```

```
// STEP 5 — Taskbar status indicators
//   All agent dots dim to 30% opacity via React state
//   MacroSentinel dot → red, full opacity, pulse animation
//   Header STATUS badge → 'HOLD' with red border

// RECOVERY: macro_status_changed { new_status: 'CLEAR' }
//   HOLD sign slides back UP — 400ms ease-in
//   Red tint fades to 0 — 300ms
//   Character returns to ACTIVE or IDLE
//   Cards unfreeze, tween resumes
//   Taskbar returns to normal colors
```

> The HOLD sign world X position is 1260 — horizontal center of the 2520-wide world space. This ensures it appears centered over the office regardless of pan position.

---

6. KILL SWITCH OPERATIONAL CENTER

A dedicated full-screen panel accessible via the /risk route or clicking Agent 7's desk. Rendered entirely as HTML overlay — not on canvas. Four tier buttons with glass cover animations, live gauges, audit trail.

6.1 Panel Layout

```
┌──────────────────────────────────────────────────────┐
│  RISK MANAGER — KILL SWITCH CONTROL                  │
├──────────────┬───────────────────────────────────────┤
│  GAUGES      │  TIER BUTTONS                         │
│              │                                       │
│  Daily       │  [TIER 1 — SOFT PAUSE    ] ARMED      │
│  Loss        │  [TIER 2 — HARD STOP     ] ARMED      │
│  ████▒       │  [TIER 3 — EMERGENCY KILL] ARMED      │
│              │  [TIER 4 — MACRO HOLD    ] MONITORING │
│  Exposure    │                                       │
│  ██████▒     │  [ MANUAL KILL — TYPE CONFIRM ]       │
├──────────────┴───────────────────────────────────────┤
│  AUDIT TRAIL  (permanent log, cannot delete)         │
│  14:32 UTC — Tier 1 auto-triggered  loss=$3.02       │
│  09:14 UTC — Manual kill  user=owner                 │
└──────────────────────────────────────────────────────┘
```

6.2 Glass Cover Animation

Each tier button has a glass cover that lifts when triggered — CSS animation only, no canvas involvement.

```
// Glass cover: a semi-transparent div positioned over each button
// On trigger: CSS transform animates cover lifting upward

@keyframes glassCoverLift {
  0%   { transform: rotateX(0deg);   opacity: 0.6; }
  100% { transform: rotateX(-90deg); opacity: 0; }
}

// Button press: scale down briefly then return
@keyframes buttonPress {
  0%   { transform: scale(1.0); }
  40%  { transform: scale(0.93); }
  100% { transform: scale(1.0); }
}

// On kill_switch_triggered event:
// 1. glassCoverLift plays on the triggered tier's cover (300ms)
// 2. buttonPress plays on the tier button (200ms)
// 3. Canvas: emergency lighting effect for severity (see below)
```

6.3 Canvas Emergency Lighting by Tier

Tier	Canvas Overlay Color	Pulse	Duration
Tier 1 — Soft Pause	rgba(0, 200, 80, 0.12)	Slow 2s pulse	Until auto-recovery
Tier 2 — Hard Stop	rgba(255, 200, 0, 0.18)	Medium 1.5s pulse	Until manual reset
Tier 3 — Emergency	rgba(255, 40, 40, 0.25)	Fast 0.8s pulse	Until manual reset
Tier 4 — Macro Hold	rgba(255, 0, 0, 0.15)	Steady (no pulse)	Until macro clears

6.4 Manual Kill Confirmation Flow

```
// Step 1: User clicks MANUAL KILL button
//   Modal appears: dark overlay + input field
//   Placeholder text: 'Type CONFIRM to proceed'

// Step 2: User types 'CONFIRM'
//   Button activates (changes from grey to red)
```

```
// Step 3: User clicks activated button
//   Second modal: 'This will close ALL positions. Are you sure?'
//   YES / NO buttons

// Step 4: YES pressed
//   POST /api/risk/kill sent to FastAPI
//   Tier 3 emergency lighting activates on canvas
//   Audit trail entry created with timestamp + 'manual' + reason field
//   All position cards in taskbar flash red then clear

// Reason field: free text input before confirmation
//   'Why are you triggering a manual kill?'
//   Logged permanently to audit trail
```

7. SIGNAL INBOX (/inbox)

Full-page HTML React component. Every signal ever processed shown as a threaded memo chain from detection to outcome. The office canvas is NOT visible on this route — it's a dedicated operations page.

7.1 Three-Panel Layout

```
┌──────────┬──────────────────────────┬──────────────┐
│  LEFT    │  CENTER                  │  RIGHT       │
│  280px   │  fills remaining         │  380px       │
│          │                          │              │
│  Agent   │  Thread index            │  Full data   │
│  filter  │  (signal list)           │  report for  │
│  list    │        OR                │  selected    │
│          │  Expanded memo thread    │  memo/signal │
│  Unread  │  when signal selected    │              │
│  badges  │                          │  Token data  │
│          │  Color-coded borders:    │  All scores  │
│  ALL     │  Green=profit            │  Trade params│
│  AGT-1   │  Red=killed              │  Outcome     │
│  AGT-2   │  Yellow=in progress      │              │
│  ...     │  Grey=expired            │              │
└──────────┴──────────────────────────┴──────────────┘
```

7.2 Memo Card Format

```
┌──────────────────────────────────────────────────────┐
```

```
│ INTERNAL MEMO                          #SIG-00847   │
│ FROM:  Agent 1 — Researcher                         │
│ TO:    Agent 2 — OnChainAnalyst                     │
│ TIME:  14:32:07 UTC  |  PRIORITY: STANDARD          │
├─────────────────────────────────────────────────────┤
│ RE: Candidate Signal — $WOJAK                       │
│ Market cap: $340k  |  Volume +840%  |  Age: 22 min  │
│ Sources: DexScreener + Researcher scan              │
│ Initial score: 5.2 — passing for verification       │
├─────────────────────────────────────────────────────┤
│  [ VIEW FULL DATA ]            [ MARK READ ]        │
└─────────────────────────────────────────────────────┘
```

7.3 Search & Filter

- Search by token name or contract address
- Filter by agent, outcome (profit/loss/killed/expired/active), date range, confidence score range
- Sort by newest, oldest, highest score, highest P&L
- Export filtered results as CSV

---

8. DEV MENU (DEVELOPER OVERLAY)

Bottom-left HTML overlay. Visible only when DEV_MODE flag is enabled. Three trigger buttons that inject mock WebSocket events to simulate market scenarios without waiting for real conditions.

8.1 Triggers

Button	Injects Events	Purpose
MARKET_CRASH	macro_status_changed {HOLD} + btc_1h_change: -6.2%	Test Tier 4 hold visual sequence
WHALE_SIGNAL	wallet_buy_detected for mock token + confidence 9.1/10	Test full signal pipeline to trade execution
EMERGENCY	kill_switch_triggered {tier: 3}	Test Tier 3 kill switch visual and panel response

```
// Dev menu fires events directly into the WebSocket event router
// Exactly as if they came from the backend — tests the full UI pipeline
```

```
function triggerDevEvent(type: 'MARKET_CRASH' | 'WHALE_SIGNAL' | 'EMERGENCY') {
  const events = DEV_EVENT_PAYLOADS[type]
  for (const event of events) {
    routeEvent(event)  // same function used by real WebSocket messages
  }
}

// Dev menu is never shipped to production — gated by:
// const DEV_MODE = import.meta.env.VITE_DEV_MODE === 'true'
```

9. WEBSOCKET → CANVAS EVENT ROUTING

Every WebSocket event maps to a specific canvas and React state change. The event router is the central nervous system — one switch statement that drives everything.

Event	Canvas Effect	HTML Overlay Effect
candidate_detected	Spawn card at agent desk, fly to table	Inbox: new thread created
wallet_buy_detected	Agent 3 URGENT anim + priority envelope	Inbox: URGENT thread, unread badge
verification_complete	Green stamp on table card	Inbox: Agent 2 reply added
signal_killed	Red X on card, slide to archive bin	Inbox: thread marked red/killed
confluence_reached	Cards combine, file to MacroSentinel	Inbox: confidence score updated
signal_expired	Cards fade + slide to archive bin	Inbox: thread marked grey
macro_status_changed HOLD	Full Tier 4 sequence (S5.2)	Header: STATUS → HOLD red
macro_status_changed CLEAR	HOLD sign retracts, tint fades	Header: STATUS → CLEAR green
kill_switch_triggered	Emergency lighting by tier (S6.3)	Kill panel: button activates
trade_executed	Agent 8 terminal flash green	Taskbar: open position card appears
take_profit_hit	Confetti particles at Agent 8 desk	Taskbar: position P&L updates
stop_loss_hit	Red flash at Agent 8 desk, slump frame	Taskbar: position card closes red
agent_status active	Agent sprite → ACTIVE, glow ON	Taskbar dot → green pulse
agent_status waiting	Agent sprite → IDLE, glow dim	Taskbar dot → grey

risk_state_update	No canvas change	Kill panel gauges update
system_health_update	All agent glow states refresh	Header: agent count update

10. BUILD PHASES

PHASE 1 — Canvas Foundation
Days 1–3

- Canvas setup with DPR handling and ResizeObserver
- Background image pre-render and cache pattern
- World-to-screen transform function
- coords.ts constants file with all coordinate map values
- Basic RAF game loop (update + render)
- Static agent sprites rendered at correct world positions
- Pan and zoom with clamped bounds

PHASE 2 — Agent State Machine
Days 4–6

- Character FSM — IDLE / ACTIVE / state-specific animations
- WebSocket connection + event router skeleton
- agent_status events driving character state changes
- Monitor glow effect per agent
- HTML taskbar with agent dots, names, status indicators
- HTML overlay labels above agents (RAF-synced positioning)

PHASE 3 — Signal Pipeline Visuals
Days 7–10

- Signal card data structure and state machine
- Card spawn at agent desk → fly to aggregator table animation
- Card stacking and score badge rendering
- Confluence reached: combine + stamp + outbound animation
- Signal expiry: fade + slide to archive bin
- Signal killed: red X shake + archive

- Agent-specific animations: URGENT envelope (Agent 3), green/red stamp (Agent 2)

PHASE 4 — MacroSentinel & Kill Switch
Days 11–13

- Sentinel wall mini-charts with threshold lines
- Tier 4 hold: full visual sequence (red tint, HOLD sign, freeze cards)
- Hold recovery sequence
- Kill switch panel (/risk route) — all four tiers
- Glass cover CSS animations
- Emergency canvas lighting by tier
- Manual kill confirmation flow
- Audit trail component

PHASE 5 — Inbox & Polish
Days 14–17

- Signal inbox (/inbox route) — three-panel layout
- Memo card format and thread color coding
- Search, filter, and sort
- Dev menu overlay with all three trigger scenarios
- Agent 8 specific: confetti particles on TP hit, slump on stop loss
- Agent 9: report print and deliver animation
- Performance pass: verify 60fps at all zoom levels
- Mobile responsive: agent card grid fallback below 768px